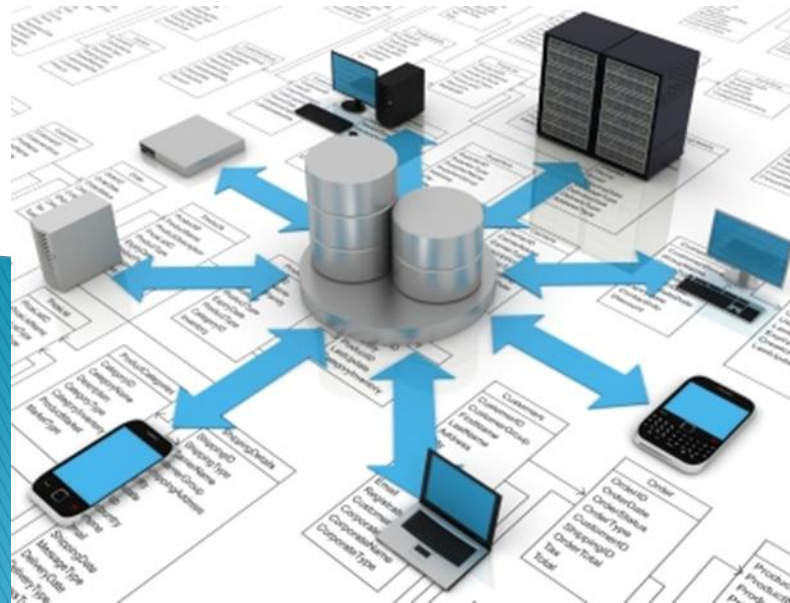
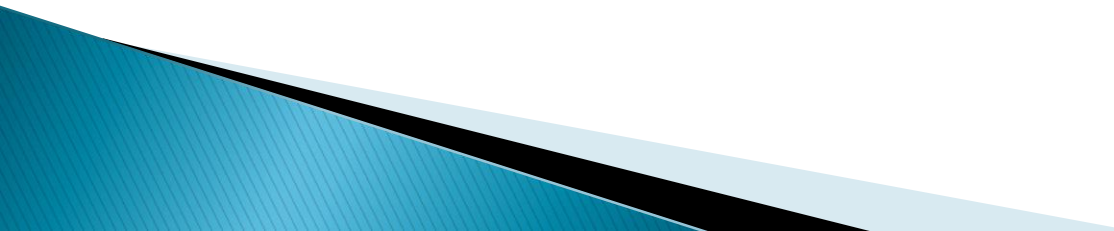


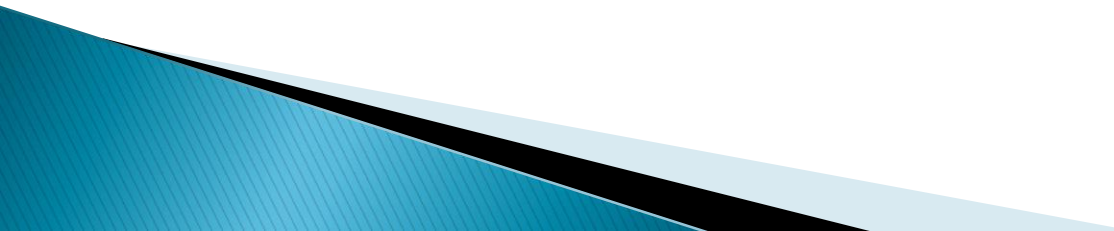
Resource Management



Resource Management

- ▶ A resource can be a logical, such as a shared file, or physical, such as a CPU (a node of the distributed system).
 - ▶ One of the functions of a DOS is to **assign processes to the nodes** of the DS to optimize
 - the resource usage,
 - response time,
 - network congestion, and
 - scheduling overhead.
- 

Desirable features of global scheduling algorithm

- ▶ No prior knowledge about the processes
 - ▶ Dynamic in nature
 - ▶ Quick decision making capability
 - ▶ Balanced system performance and scheduling overhead
 - ▶ Stability (Processor thrashing must be prevented [due to migration, no node should be in idle state])
 - ▶ Scalability
 - ▶ Fault tolerance
- 

Techniques for process Scheduling

- ▶ There are 3 techniques for scheduling processes of a DS:
 - 1) *Task Assignment Approach*, in which each **submitted process** is viewed as a collection of related tasks and these tasks are **assigned to suitable nodes** to improve performance.
 - 2) *Load-balancing approach*, in which all the **submitted processes** are distributed among the nodes of the system **to equalize the workload** among the nodes.
 - 3) *Load-sharing approach*, which simply attempts to conserve the ability of the system to perform work by assuring that **no node is idle** while processes wait for being processed.
- 

Task Assignment Approach

- ▶ The task assignment approach has limited applicability to practical situations since it works on the assumption that the characteristics (e.g. execution time, IPC costs etc) of all the processes to be scheduled are **known in advance**.

Task Assignment Approach

- ▶ Assumptions:

- 1) A process has already been split up into *tasks*.

This split occurs along natural boundaries (such as a method), so that each task will have integrity in itself and data transfers among the tasks are minimized.

- 2) The amount of computation required by each task and the speed of each CPU are known.

- 3) The cost of processing each task on every node is known. This is derived from assumption 2.

Task Assignment Approach

- ▶ Assumptions:

4) The IPC costs between every pair of tasks is known.

- ▶ The IPC cost is zero for tasks assigned to the same node.
- ▶ This is usually estimated by an analysis of the static program.
- ▶ If two tasks communicate n times and the average time for each inter-task communication is t , then IPC costs for the two tasks is $n * t$.

5) Precedence relationships among the tasks are known.

6) Reassignment of tasks is not possible.

Task Assignment Approach

- ▶ Aim is to assign the tasks of a process to the nodes to achieve the following goals:
 - Minimization of IPC costs
 - Quick turnaround time for the complete process
 - A high degree of parallelism
 - Efficient utilization of system resources in general

These goals often conflict.

- E.g., while minimizing IPC costs tends to assign all tasks of a process to a single node, affects the efficient utilization of system resources.
- Quick turnaround time and a high degree of parallelism encourage parallel execution of the tasks, the precedence relationship among the tasks limits their parallel execution.

Task Assignment Approach

Note:

- in case of t tasks and n nodes, there are n^t possible assignments of tasks to nodes.
- In practice the actual number of possible assignments of tasks to nodes may be less than n^t due to the restriction that certain tasks cannot be assigned to certain nodes due to their specific requirements (e.g. need a certain amount of memory or a certain data file).

Task assignment example

- There are two nodes, $\{p_1, p_2\}$ and six tasks $\{t_1, t_2, t_3, t_4, t_5, t_6\}$. There are two task assignment parameters – the task execution cost (x_{tn} the cost of executing task t on node n) and the inter-task communication cost (c_{ij} the inter-task communication cost between tasks i and j).

Inter-task communication cost

	t1	t2	t3	t4	t5	t6
t1	0	6	4	0	0	12
t2	6	0	8	12	3	0
t3	4	8	0	0	11	0
t4	0	12	0	0	5	0
t5	0	3	11	5	0	0
t6	12	0	0	0	0	0

Execution costs

	<u>Nodes</u>	
	p1	p2
t1	5	10
t2	2	∞
t3	4	4
t4	6	3
t5	5	2
t6	∞	4

Task t_6 cannot be executed on node p_1 and task t_2 cannot be executed on node p_2 since the resources they need are not available on these nodes.

Task assignment example

- 1) **Arbitrary assignment**, where tasks t_1, t_2, t_3 are assigned to node p_1 and tasks t_4, t_5, t_6 are assigned to node p_2 :

$$\text{Execution cost, } x = x_{11} + x_{21} + x_{31} + x_{42} + x_{52} + x_{62} = 5 + 2 + 4 + 3 + 2 + 4 = 20$$

$$\begin{aligned}\text{Communication cost, } c &= c_{14} + c_{15} + c_{16} + c_{24} + c_{25} + c_{26} + c_{34} + c_{35} + c_{36} \\ &= 0 + 0 + 12 + 12 + 3 + 0 + 0 + 11 + 0 = 38.\end{aligned}$$

Hence total cost = 58.



- 2) **Optimal assignment**, where tasks t_1, t_2, t_3, t_4, t_5 are assigned to node p_1 and task t_6 is assigned to node p_2 .

$$\begin{aligned}\text{Execution cost, } x &= x_{11} + x_{21} + x_{31} + x_{41} + x_{51} + x_{62} \\ &= 5 + 2 + 4 + 6 + 5 + 4 = 26\end{aligned}$$

$$\begin{aligned}\text{Communication cost, } c &= c_{16} + c_{26} + c_{36} + c_{46} + c_{56} \\ &= 12 + 0 + 0 + 0 + 0 = 12\end{aligned}$$

Total cost = 38

Communication Cost

	t1	t2	t3	t4
t1	0	35	3	8
t2	35	0	6	4
t3	3	6	0	23
t4	8	4	23	0

Execution Cost

	p1	p2
t1	31	4
t2	1	5
t3	2	4
t4	3	23

Example

Find the optimal
Assignment

Assignment - 1

t	p
t1	p2
t2	p2
t3	p2
t4	p1

$$\begin{aligned} EC &= 4 + 5 + 4 + 3 = 16 \\ CC &= 8 + 4 + 23 = 35 \\ TC &= 16 + 35 = 51 \end{aligned}$$

Assignment - 2

t	p
t1	p2
t2	p1
t3	p1
t4	p1

$$\begin{aligned} EC &= 4 + 1 + 2 + 3 = 10 \\ CC &= 35 + 3 + 8 = 46 \\ TC &= 10 + 46 = 56 \end{aligned}$$

Assignment - 3

t	p
t1	p2
t2	p1
t3	p2
t4	p1

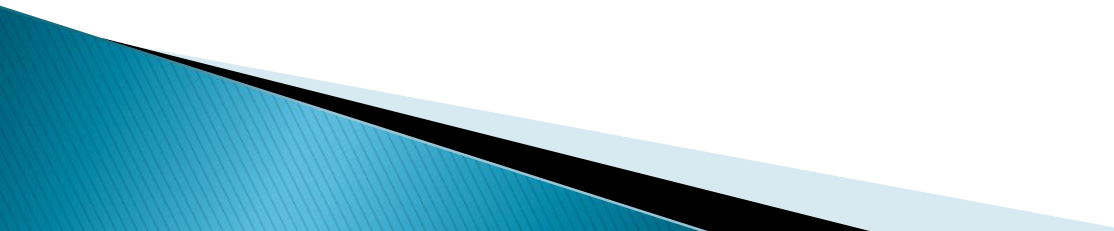
$$\begin{aligned} EC &= 4 + 4 + 1 + 3 = 12 \\ CC &= 35 + 8 + 6 + 23 = 72 \\ TC &= 12 + 72 = 84 \end{aligned}$$

Assignment - 4

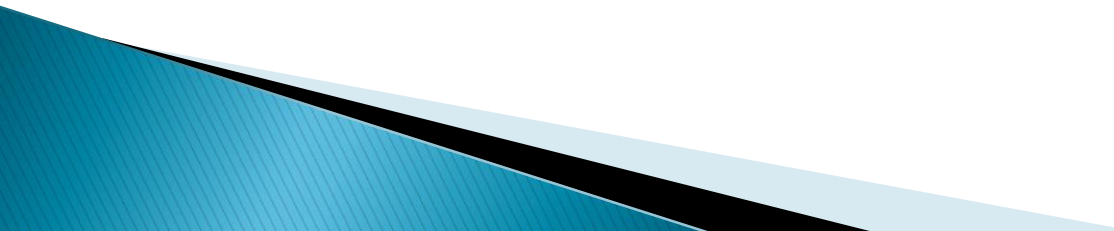
t	p
t1	p2
t2	p2
t3	p1
t4	p1

$$\begin{aligned} EC &= 4 + 5 + 2 + 3 = 14 \\ CC &= 3 + 6 + 8 + 4 = 21 \\ TC &= 14 + 21 = 35 \end{aligned}$$

Load-balancing approach

- ▶ Processors in a system can be identified according to their present load.
 - Heavily loaded processors where enough jobs are waiting for execution;
 - Lightly loaded processors where less jobs are waiting;
 - Idle processors which have no job to execute.
 - ▶ The basic aim is to make every processor **equally busy** and to **finish the works** approximately **at the same time**
 - ▶ This can be achieved by the load balancing concept.
- 

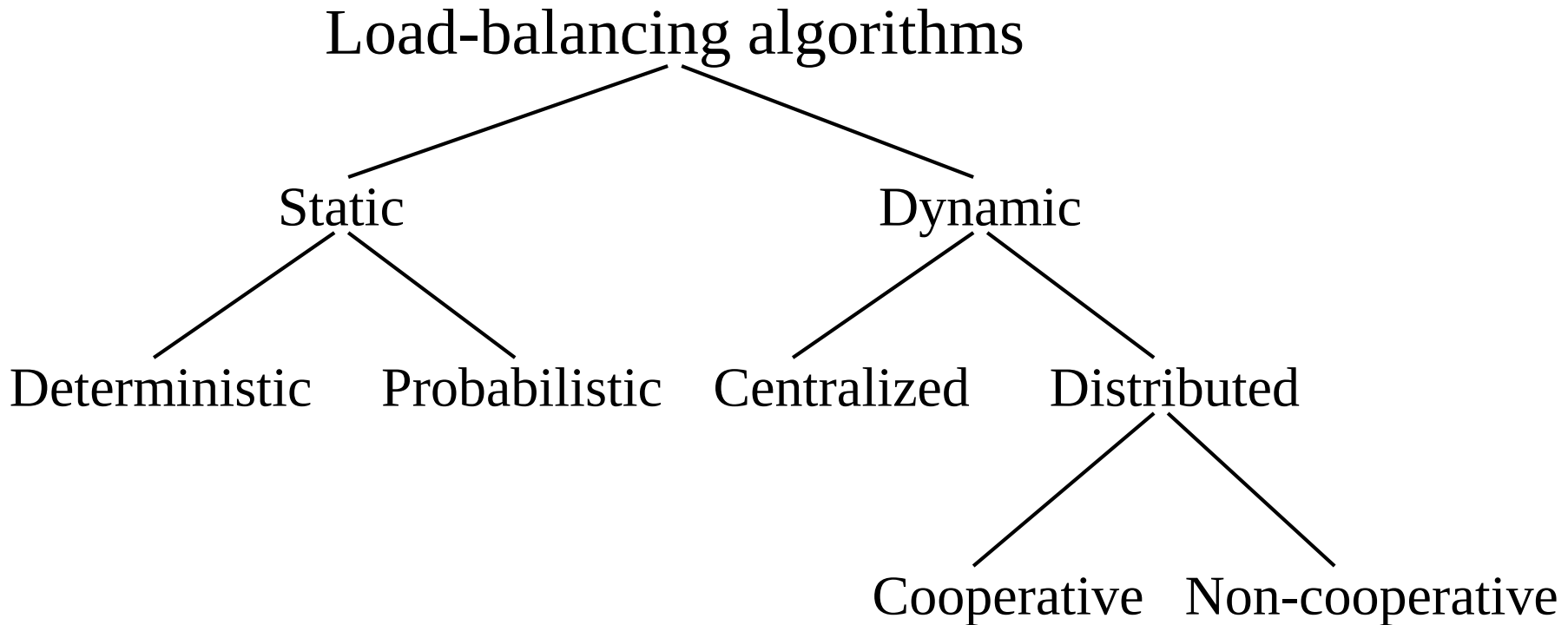
Load-balancing approach

- ▶ In the load balancing operation, we have three rules: location rule, distribution rule and selection rule.
 - ▶ Practically load balancing decisions are taken jointly by location and distribution rules
 - ▶ The selection rule will work either in pre-emptive or in non-pre-emptive fashion.
 - The newly generated process is always picked up by the non-pre-emptive rule while the running process may be picked up by the pre-emptive rule.
 - It is observed that pre-emptive transfer is a costly as compared to the non-pre-emptive transfer that is more preferable.
- 

Benefits of Load-balancing

- ▶ Improves the performance of each node and hence the overall system performance.
 - ▶ It reduces the job idle time.
 - ▶ Small jobs do not suffer from long starvation.
 - ▶ It makes maximum utilization of resources.
 - ▶ Response time becomes shorter.
 - ▶ It gives higher throughput.
 - ▶ It gives higher reliability.
 - ▶ It has low cost and high gain.
 - ▶ It gives extensibility and incremental growth.
- 

Load-balancing approach



A Taxonomy of Load-Balancing Algorithms

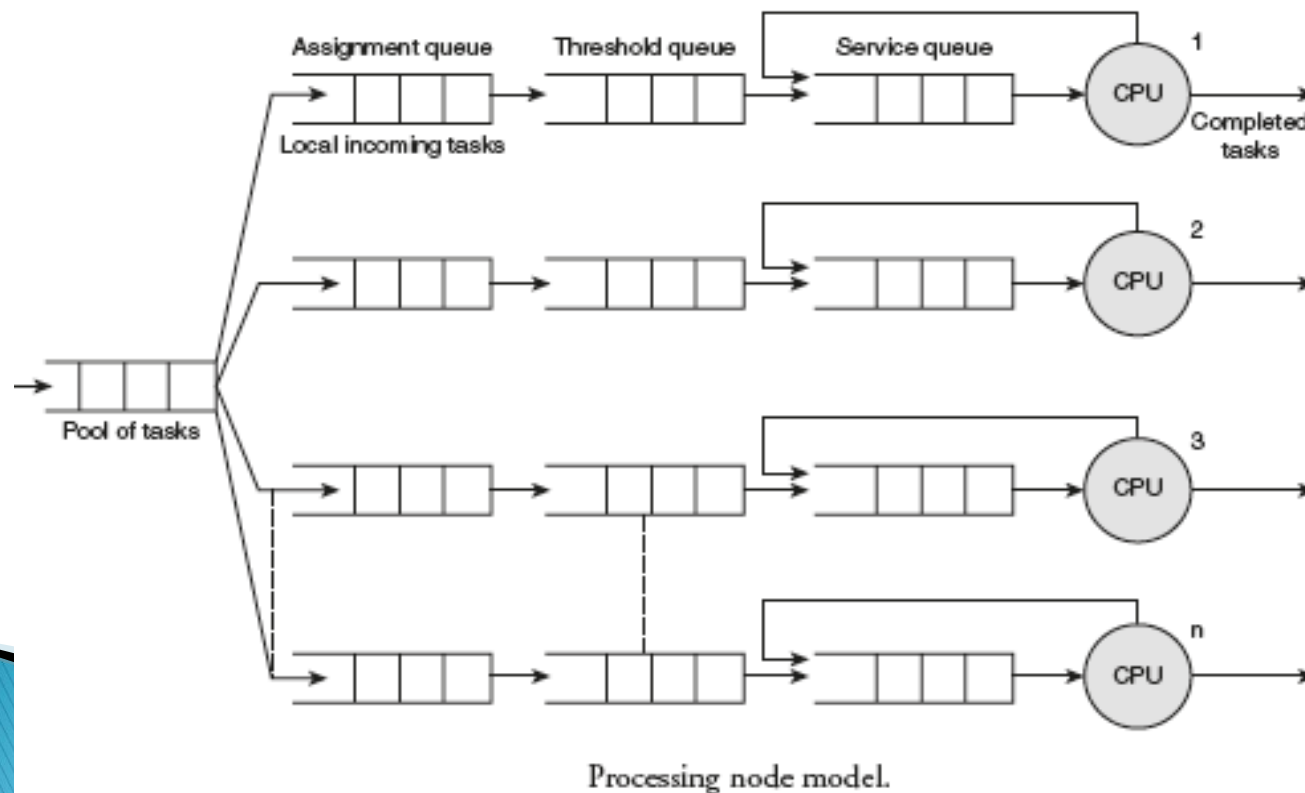
Static Load Balancing

- ▶ Where, the **processes** are assigned to the **processors** at the **compile time**, depending on the performance of the nodes.
 - ▶ After the assignment of processes, no change or reassignment is possible at the runtime.
 - ▶ The total number of jobs in each node is fixed.
 - ▶ The information about the node is not collected.
 - ▶ The job assignment is done to the processing nodes on the basis of the following factors:
 1. Incoming time.
 2. Extent of resource needed.
 3. Mean execution time.
 4. Inter-process communications.
- 

Static Load Balancing

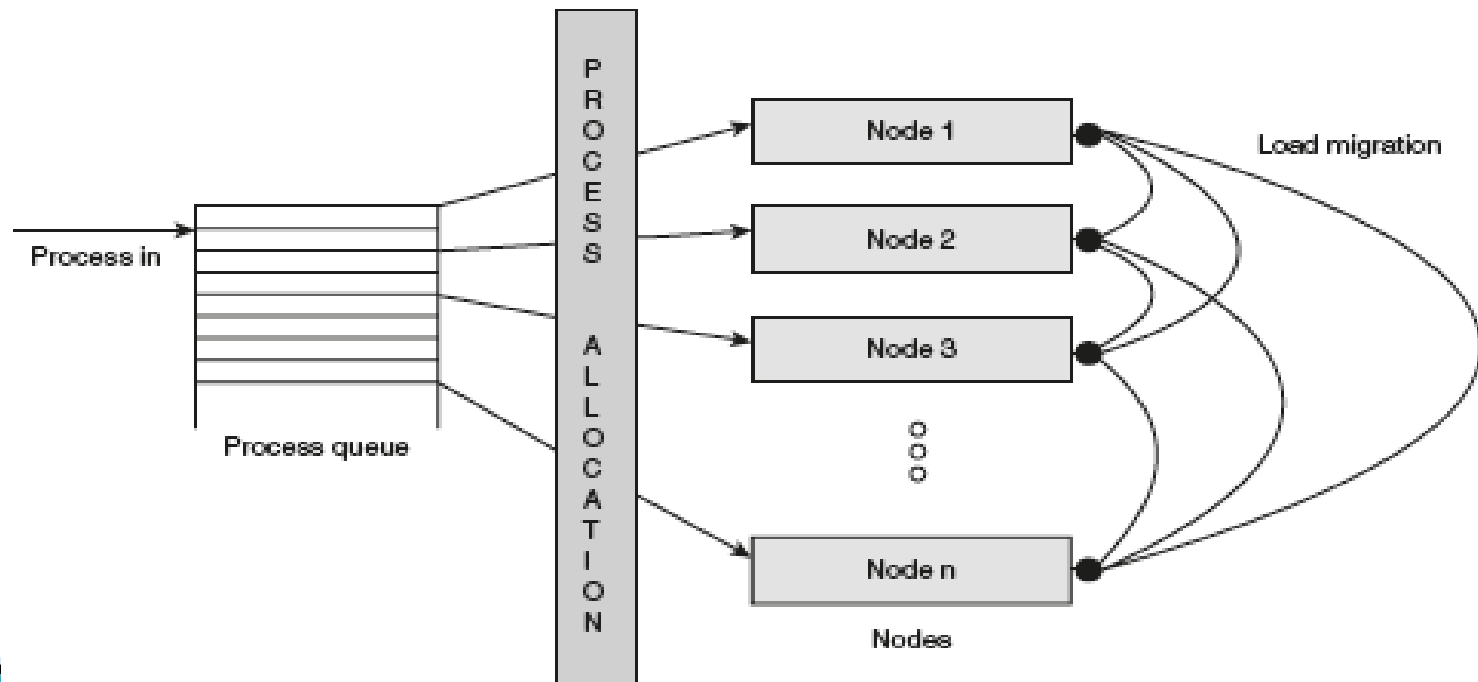
Drawbacks:

- ▶ Information about the system and jobs must be known before the execution.
- ▶ It is very difficult to estimate execution times of various parts of a program accurately
- ▶ Communication delay may occur under different circumstances



Dynamic Load Balancing

- ▶ The assignment of jobs is done at the **runtime** and jobs are reassigned at the runtime, depending upon the situation, that is, the load will be transferred from heavily loaded nodes to lightly loaded nodes.
- ▶ Communication overhead occurs and becomes more when the number of processors increases.



Simple dynamic load balancing existence.

Static verses dynamic

- ▶ Static algorithms use only information about the **average behavior of the system, ignoring the current state** of the system.
 - **simpler** because there is **no need to maintain state information** of process.
 - static algorithms is limited by the fact that they do not react to the current system state.
 - Low performance compared to dynamic.
 - ▶ Dynamic algorithms **react to the system state** that changes dynamically
 - **Complex** because they **collect and react to system state information**
 - they do respond to system state and so are better able to avoid those states with unnecessarily poor performance
 - provide greater performance benefits
- 

Comparison between Static and Dynamic Load Balancing

Factor	Load Balancing	
	SLB Algorithm	DLB Algorithm
Nature	Work load is assigned at compile time	Work load is assigned at run time
Overhead involved	Little overhead due to IPC	Greater overhead due to process redistribution
Resource utilization	Lesser utilization	Greater utilization
Predictability	Easy to predict	Difficult to predict
Adaptability	Less adaptive	More adaptive
Reliability	Less	More
Response time	Short	Longer
Stability	More	Less
Complexity	Less	More
Cost	Less	More

Deterministic versus Probabilistic

- ▶ Deterministic algorithms use the information about the **properties of the nodes** and the **characteristics of the processes** to be scheduled to allocate processes to nodes.
 - ▶ The **task assignment algorithms** belong to this category.
 - ▶ A probabilistic load-balancing algorithm uses information regarding **static attributes of the system** to formulate simple process placement rules.
 - number of nodes, the processing capability of each node, the network topology, and so on
 - ▶ The deterministic approach is difficult to optimize and costs more to implement.
 - ▶ The probabilistic approach is easier to implement but often suffers from having poor performance.
- 

Centralized versus Distributed

- ▶ In a centralized algorithm, the responsibility of scheduling resides on a single node.
 - In the centralized approach, the system state information is collected at a single node at which all scheduling decisions are made.
- ▶ In a distributed dynamic scheduling algorithm, the work involved in making process assignment decisions is physically distributed among the various nodes of the system.
 - A distributed scheme does not limit the scheduling intelligence to 1 node.
 - It avoids the bottleneck of collecting state information at a single node and allows the scheduler to react quickly to dynamic changes in the system.
 - A distributed dynamic scheduling algorithm is composed of k physically distributed entities $e_1, e_2, \dots, e_k$. Each entity is considered a local controller.

Cooperative versus Non-cooperative

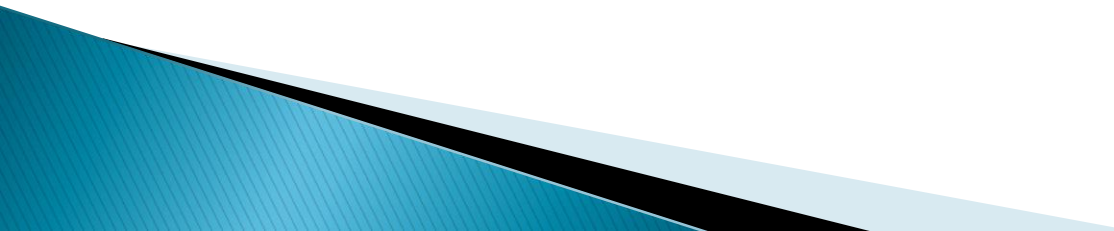
- ▶ In non-cooperative algorithms, individual entities act as autonomous entities and make scheduling decisions independently of the actions of other entities.
- ▶ In cooperative algorithms, the distributed entities cooperate with each other to make scheduling decisions.
 - Cooperative algorithms are more complex and involve larger overhead than non-cooperative ones.
 - The stability of a cooperative algorithm is better than that of a non-cooperative algorithm.

Issues In Designing load-balancing Algorithms

- ▶ **Load estimation policy**, which determines how to estimate the workload of a particular node of the system
 - ▶ **Process transfer policy**, which determines whether to execute a process locally or remotely
 - ▶ **State information exchange policy**, which determines how to exchange the system load information among the nodes
 - ▶ **Location policy**, which determines to which node a process selected for transfer should be sent
 - ▶ **Priority assignment policy**, which determines the priority of execution of local and remote processes at a particular node
 - ▶ **Migration limiting policy**, which determines the total number of times a process can migrate from one node to another
- 

Load estimation policy

- ▶ To decide the method to be used to estimate the workload of a particular node.
 - No completely satisfactory solution exists. Can be estimated based on some measurable parameters.
 - These parameters could include **time dependent and node-dependent** factors such as:
 - **Total number of processes** on the node at the time of load estimation
 - **Resource demands** of these processes
 - **Instruction mixes** of these processes
 - **Architecture and speed** of the node's processor
 - The **true load** could vary depending on the remaining **service times** for those processes.
 - For estimating a node's workload it is required to find the sum of the remaining service times of all the processes on that node. (**Issue is how to estimate the remaining service time of the processes**)

- ▶ No method is suitable for use as load estimation policies in modern distributed systems since several processes (such as mail and news daemons, window managers) exist permanently, even on an idle node.
 - ▶ Many of these processes wake up periodically, check to see if there is anything that they have to do, and if not, go back to sleep.
 - ▶ The method for use as the load estimation policy in these systems would be to **measure the CPU utilization of the nodes**.
 - ▶ Central processing unit (CPU) utilization is defined as the number of CPU cycles actually executed per unit of real time.
 - ▶ The CPU utilization of a node can be measured by **setting up a timer** to periodically observe the CPU state.
- 

Process Transfer Policies

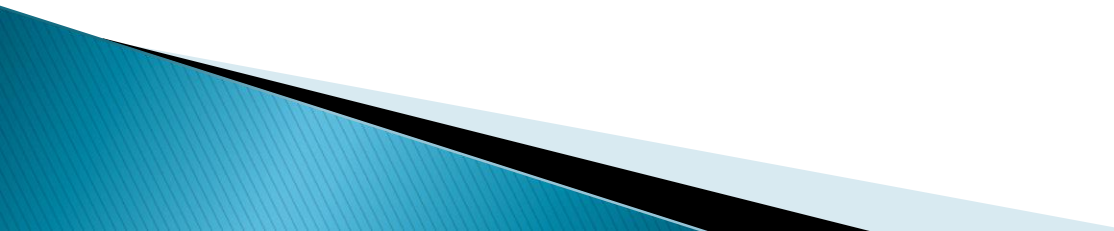
- ▶ Use the *threshold policy* to make this decision.
- ▶ The threshold value of a node is the limiting value of its workload and is used to decide whether a node is *lightly or heavily loaded*.
 - A new process at a node is accepted locally for processing if the workload of the node is below its threshold value at that time.
 - Otherwise, an attempt is made to transfer the process to a lightly loaded node.
- ▶ The threshold value of a node may be determined by any of the following methods:
 - ***Static policy***: each node has a predefined threshold value depending on its processing capability.
 - ***Dynamic policy***: the threshold value of a node (ni) is calculated as a product of the average workload of all the nodes and a predefined constant (ci).
 - For each node ni , the value of ci depends on the processing capability of that node, relative to the processing capability of an other nodes.

Location Policies

- ▶ To select the destination node for the process's execution the location policy of a scheduling algorithm is used.
- ▶ The main location policies are: Threshold, Shortest, Bidding.
- ▶ **Threshold:** a destination node is selected at random and a check is made
 - To determine whether the transfer of the process to that node would place it in a state that prohibits the node to accept remote processes.
 - If not, the process is transferred to the selected node, which must execute the process regardless of its state when the process actually arrives.
 - On the other hand, another node is selected at random and probed in the same manner.
 - This continues until either a suitable destination node is found or the number of probes exceeds a static probe limit L_p .

- ▶ **Shortest:** In this method, *distinct nodes are chosen at random*, and each is polled in turn to determine its load.
 - The process is transferred to the node having the minimum load value.
 - If none of the polled nodes can accept the process, it is executed at its originating node.
 - If a destination node is found and the process is transferred there, the destination node must execute the process.
- ▶ **Improvement:** discontinue probing whenever a node with zero load is encountered, since that node is guaranteed to be an acceptable destination.

Bidding: In this method, the system is turned into a distributed computational economy with **buyers and sellers of services**.

- ▶ Each node in the network is responsible for two roles with respect to the bidding process: manager and contractor.
 - The *manager* represents a node having a *process in need of a location to execute*, and
 - The *contractor* represents a node that is *able to accept remote processes*.
 - ▶ The manager broadcasts a **request-for-bids** message to all other nodes in the system.
 - ▶ Then the contractor nodes **return bids** to the manager node.
 - The bids contain the processing capability, memory size, resource availability of the contractor nodes.
 - ▶ After received bids from the contractor nodes, the manager node chooses the best bid and the process is transferred from the manager node to the winning contractor node.
- 

State Information Exchange Policies

- ▶ The dynamic policies require frequent exchange of state information among the nodes of the system.
- ▶ The proper selection of the state information exchange policy is essential.
- ▶ The proposed load-balancing algorithms use one of the following policies for this purpose:
 - **Periodic Broadcast**
 - **Broadcast When State Changes**
 - **On-Demand Exchange** (*A node needs to know about the state of other nodes only when it is either underloaded or overloaded, broadcasts *StateInformationrequest* message*)
 - **Exchange by Polling** (*randomly polling the other nodes one by one, state information is exchanged only between the polling node and the polled nodes.*)

Priority Assignment Policies

One of the following priority assignment rules may be used:

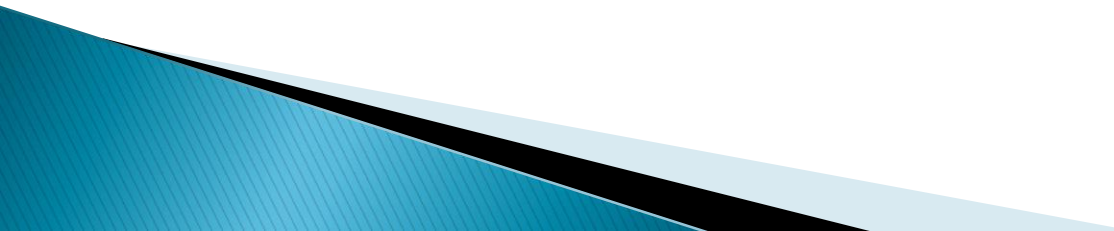
1. **Selfish.** Local processes are given higher priority than remote processes (worst response time performance)
2. **Altruistic.** Remote processes are given higher priority than local processes (best response time performance).
3. **Intermediate.** The priority of processes depends on the number of local processes and remote processes at the concerned node.
 - If the number of local processes $\geq$ remote processes, local processes are given higher priority than remote processes.
 - Otherwise, remote processes are given higher priority than local processes (fall between above two).

Migration-Limiting Policies

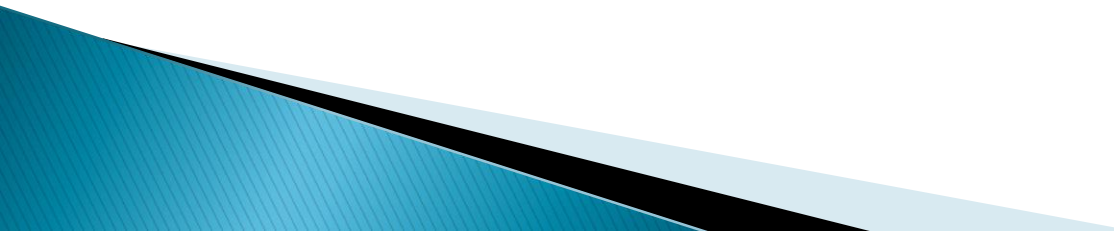
One of the following two policies may be used:

1. *Uncontrolled:* In this case, a remote process arriving at a node is treated just as a process originating at the node.
 - Therefore a process may be migrated any number of times.
 - This policy has the unfortunate property of causing instability.
2. *Controlled:* To overcome the instability problem most systems treat remote processes different from local processes and use a *migration count* parameter to fix a limit on the number of times that a process may migrate.

Load Balancing Techniques

- ▶ Round Robin
 - ▶ Weighted Round Robin
 - ▶ Least Connection
 - ▶ Weighted Least Connection
 - ▶ Resource Based (Adaptive)
 - ▶ Fixed Weighting
 - ▶ Weighted Response Time
- 

Load-sharing Approach

- ▶ For the proper utilization of the resources of a distributed system, it is necessary and sufficient to prevent the nodes from being idle while some other nodes have more than two processes.
 - ▶ Therefore this rectification is often called *dynamic load sharing instead of dynamic load balancing*.
 - ▶ Issues in Designing load-Sharing Algorithms
 - Load Estimation Policies
 - Process Transfer Policies
 - Location Policies
 - State Information Exchange Policies
- 

Issues in Designing load-Sharing Algorithms

▶ Load Estimation Policies:

- To ensure that no node is idle while processes wait for service at some other node, it is sufficient to know whether a node is busy or idle.

▶ Process Transfer Policies

- Load-sharing algorithms are normally interested only in the busy or idle states of a node, most of them employ the *all-or-nothing strategy*.
 - This strategy uses the single threshold policy with the threshold value of all the nodes fixed at 1.
 - That is, a node becomes a candidate for accepting a remote process only when it has no process, and a node becomes a candidate for transferring a process as soon as it has more than one process.

Issues in Designing load-Sharing Algorithms

▶ Location Policies:

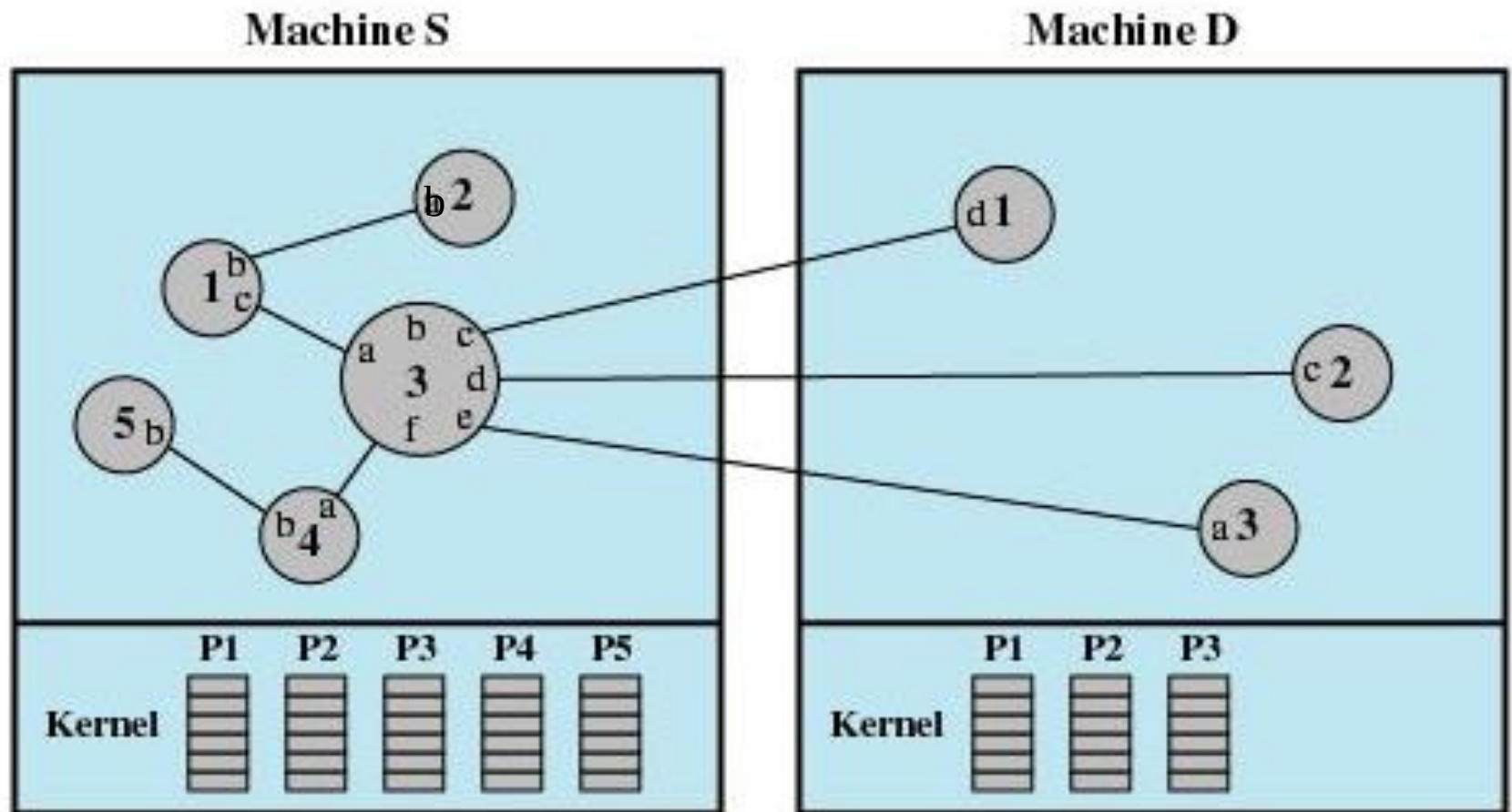
- ***Sender-Initiated Location Policy:*** in which the sender node of the process decides where to send the process
- ***Receiver-Initiated Location Policy:*** in which the receiver node of the process decides from where to get the process

▶ State Information Exchange Policies

- ***Broadcast When State Changes:*** In this method, a node broadcasts a *StateInformationRequest* message when it becomes either under loaded or overloaded.
- ***Poll When State Changes:*** In this method, when a node's state changes, it does not exchange state information with all other nodes but randomly polls the other nodes one by one and exchanges state information with the polled nodes.

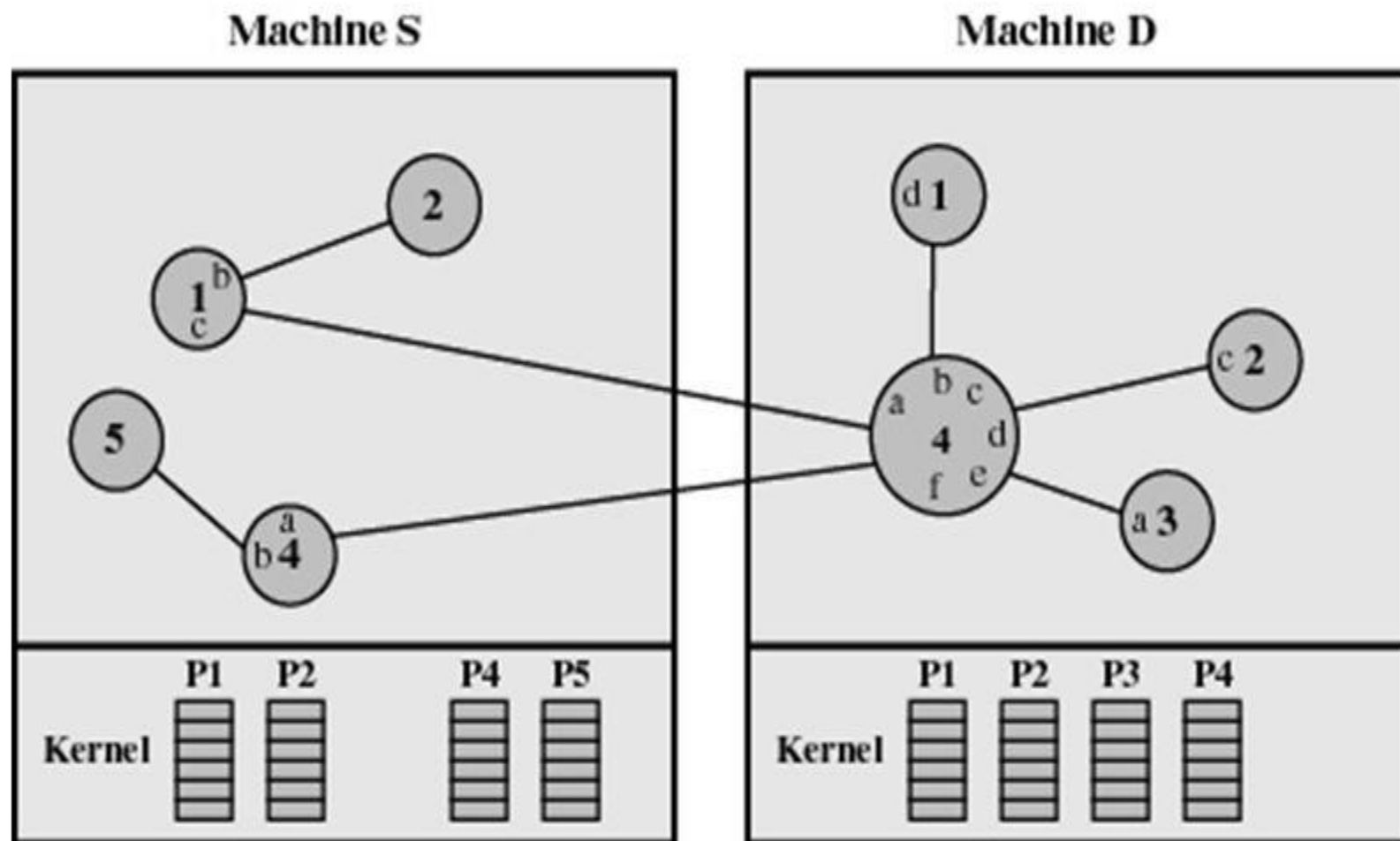
PROCESS MANAGEMENT

Example of process migration



(a) Before migration

Implementation of process migration (cont.)



(b) After migration

Introduction

- ▶ The main goal of process management is to make the best possible use of the processing resources of the entire system by sharing them among all processes.
- ▶ Three important concepts to achieve this goal:
 - ▶ **Processor Allocation:** deals with the process of deciding which process should be assigned to which processor.
 - ▶ **Process/Code Migration:** deals with the movement of a process from its current location to the processor to which it has been assigned.
 - ▶ **Threads:** deals with fine-grained parallelism for better utilization of the processing capability of the system

Process Migration

- ▶ **Process Migration** is the relocation of a process from its current location to another location. This involves the following steps:
 - ▶ Selection of a process that should be migrated.
 - ▶ Selection of the destination node.
 - ▶ Actual transfer of the selected process to the destination node.
- ▶ A process may be migrated either **before it starts executing** on its source node (*non-preemptive process migration*) or **during the execution** (*preemptive process migration*).
- ▶ The first two steps are taken care of by the **process migration policy** and the third step is taken care of by the **process migration mechanism**.

Process Migration

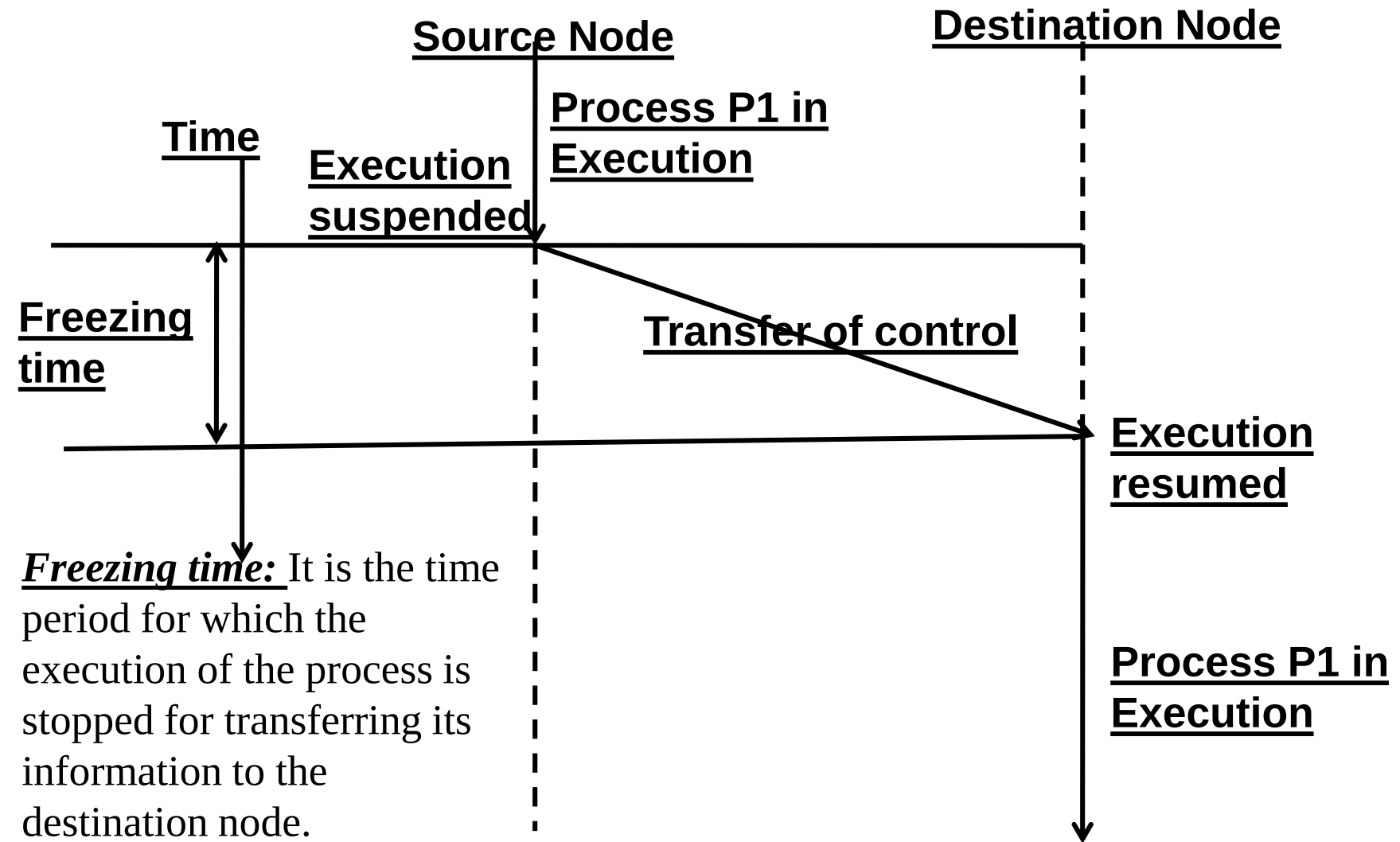


Figure : Flow of execution of a migrating process

Desirable Features of a Good Process Migration

- ▶ **Transparency:**
 - ▶ **Object access level** - access to objects (such as files and devices) by process can be done in location-independent manner.
 - ▶ **System call and interprocess communication level** - the communicating processes should not notice if one of the processes is moved to another node, system calls should be **equivalent**.
- ▶ **Minimal interference** (with process execution) - minimize freezing time
- ▶ **Minimal residual dependencies** - a migrated process should not continue to depend on its previous node once it has started executing on its new node. Otherwise,
 - ▶ The migrated process continues to impose a load on its previous node
 - ▶ A failure or reboot of the previous node will cause the process to fail.

Desirable Features of a Good Process Migration

- ▶ **Efficiency:** the following costs should be kept to minimum.
 - ▶ The time required for migrating a process
 - ▶ The cost of locating an object (includes the migrated process)
 - ▶ The cost of supporting remote execution once the process is migrated
- ▶ **Robustness**
 - ▶ the failure of a node other than the one on which a process is currently running should not affect the accessibility or execution of that process.
- ▶ **Communication between Co-processes of a Job**
 - ▶ If a single job is distributed over several nodes, then these co-processes must be able to directly communicate with each other irrespective of their locations.

Process Migration Mechanisms

- ▶ The four major sub activities involved in process migration are as follows:
 1. Freezing the process on its source node and restarting on its destination node
 2. Transferring the process's address space
 3. Forwarding messages meant for the migrant process
 4. Handling the communication between cooperating processes that have been separated.

1. Mechanisms for freezing and restarting a process

- ▶ The usual process of migration is to take a “snapshot” of the process’s state on its source node and reinstate the snapshot on the destination node.
- ▶ For this, at some point during migration,
 - ▶ the process is frozen on its source node,
 - ▶ its state information is transferred to its destination node,
 - ▶ the process is restarted on its destination node using this state information.
- ▶ Freezing the process means, the execution of the process is suspended and all external interactions with the process are deferred.

Issues involved in freezing and restarting a process

- ▶ Immediate and Delayed Blocking of the process
- ▶ Fast and slow I/O operations: Finish fast I/O
- ▶ Information about open Files
- ▶ Reinstating the process on its destination node

- ▶ **Blocks the execution of the migrant process, postponing all external communication**
 - **immediate blocking** - when not executing system call
 - **postponed blocking** - when executing certain system calls
- ▶ **Wait for I/O operations:**
 - wait for fast I/O before freezing the process: example disk I/O
 - arrange to gracefully resume slow I/O operations at destination: terminal I/O, network communication
- ▶ **Takes a “snapshot” of the process state**
 - **relocatable information** - register contents, program counter, etc.
 - **Open files information** - names, identifiers, access modes, current positions of file pointers, etc.
- ▶ **Transfers the process state to the destination**
- ▶ **Restarts the process on destination node**

▶ Reinstating the process on its destination node

- ▶ On the destination node, an empty process state is created similar to that allocated during process creation.
- ▶ The newly created copy of the process initially has a **process identifier different from the migrating process**.
- ▶ This allows both the old and new copies to exist and be accessible at the same time.
- ▶ If the process identifier of the new copy is different from its old copy, the new copy's identifier is changed to the original identifier before the process starts executing on the destination node.
- ▶ After all the state of the migrating process has been transferred and copied into the empty process state, the new copy of the process is unfrozen and the old copy is deleted.

2. Address space transfer mechanism

- ▶ The migration of a process involves the transfer of the following information :

- **Process's state:**

- the execution status (contents of registers),
- scheduling information,
- information about main memory being used by the process (memory tables),
- I/O states (I/O queue, contents of I/O buffers, interrupt signals, etc.),
- a list of objects to which the process has a right to access,
- process's identifier,
- process's user and group identifiers,
- information about the files opened by the process (such as the mode and current position of the file pointer), and so on

- **Process's address space** (code, data, and stack of the program)

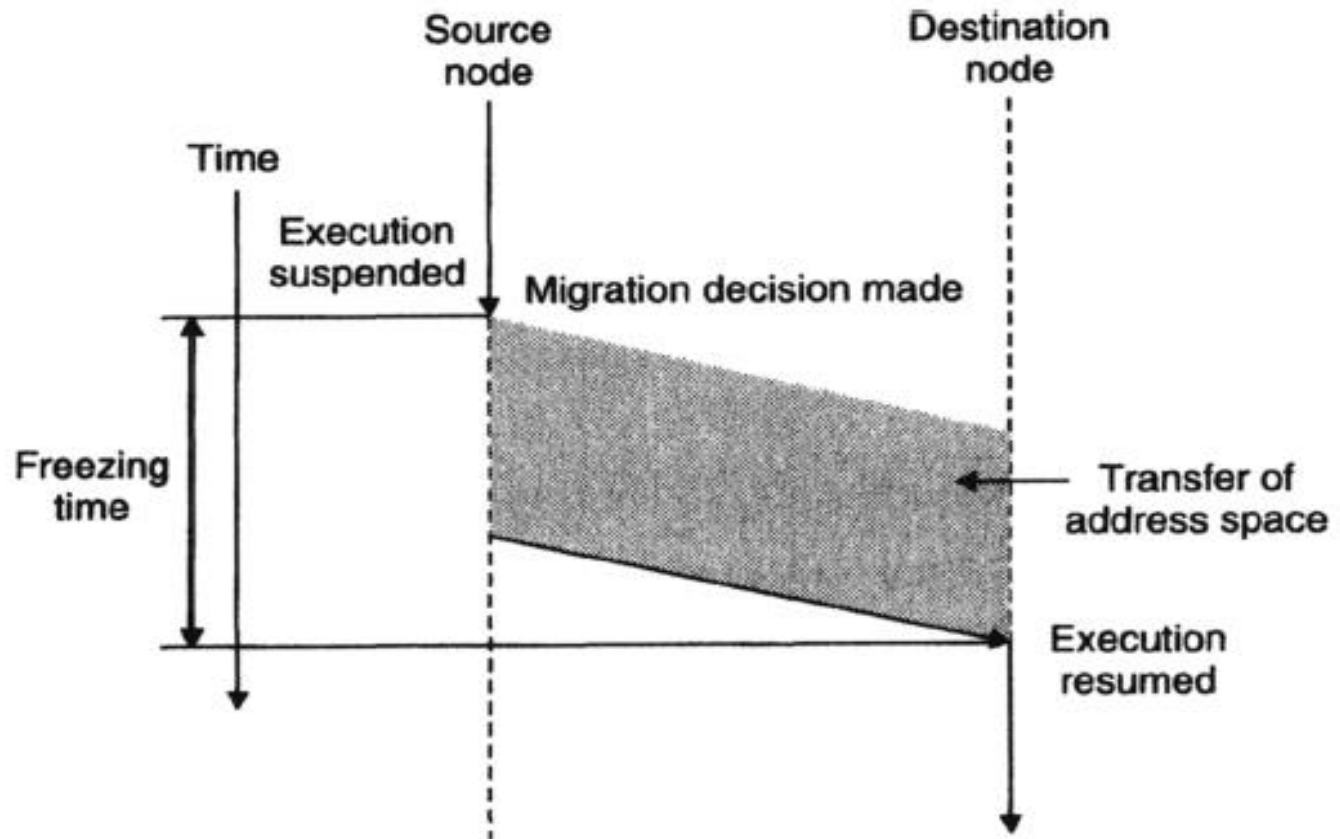
- ▶ It is necessary to completely stop the execution of the migrant process while transferring its state information.

2. Address space transfer mechanism...

- ▶ The process's address space can be transferred to the destination node either before or after the process starts executing on the destination node
- ▶ The address space transfer mechanisms:
 - ▶ Total freezing
 - ▶ Pretransferring
 - ▶ Transfer on Reference

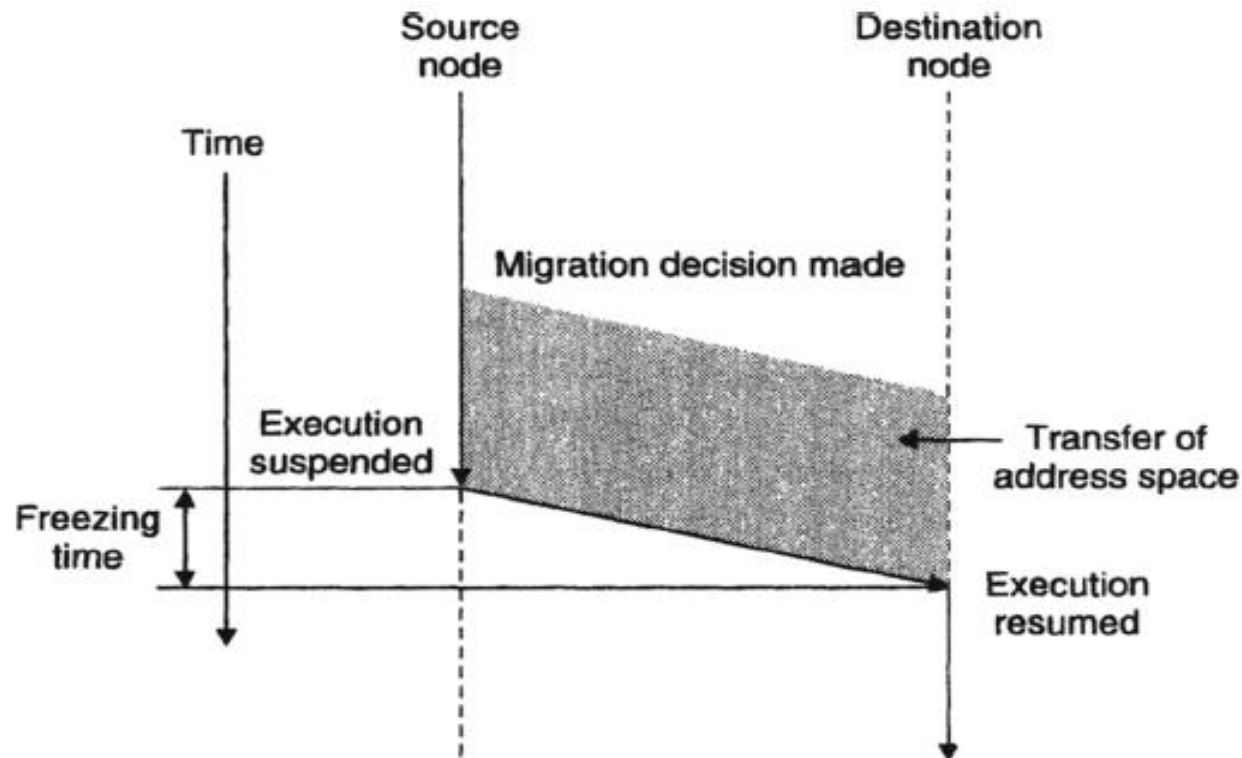
i. Total Freezing

- ▶ A **process's execution** is stopped while its address space is being transferred
- ▶ It is simple and easy to implement.
- ▶ Its main disadvantage is that if a process is suspended for a long time during migration, timeouts may occur, and if the process is interactive, the delay is more.



ii. Pretransferring

- ▶ The address space is transferred while the process is still running on the source node.
- ▶ Once the decision has been made to migrate a process, it continues to run on its source node until its address space has been transferred to the destination node.
- ▶ The remaining modified pages are retransferred after the process is frozen for transferring its state information.

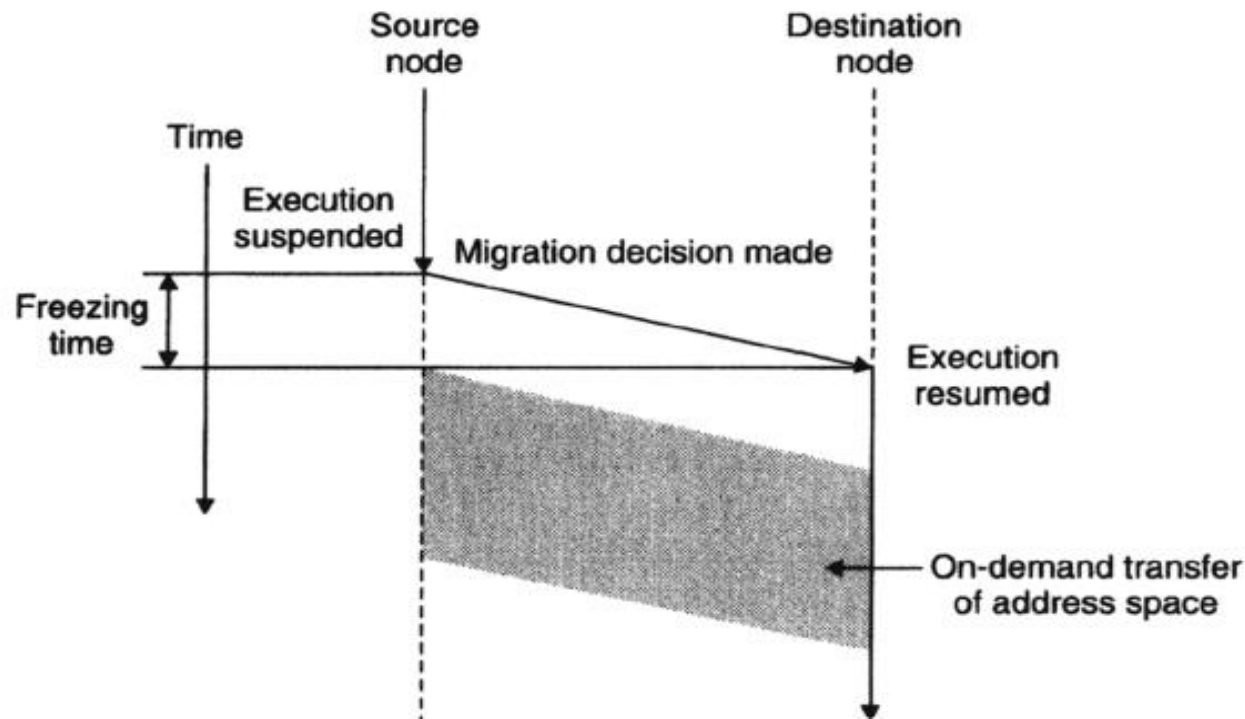


ii. Pretransferring

- ▶ In the pretransfer operation,
 - the first transfer operation moves the entire address space and takes the longest time, thus providing the **longest time for modifications to the program's address space** to occur.
 - The second transfer moves only those pages of the address space that were **modified during the first transfer**, thus taking **less time** and presumably allowing fewer modifications to occur during its execution time.
 - Then subsequent transfer operations have to move **fewer and fewer** pages, finally converging to zero or very few pages, which are then transferred after the process is frozen.
- ▶ The pretransfer operation is executed at a higher priority than all other programs to prevent the other programs from interfering with the progress of the pretransfer operation.

iii. Transfer on Reference

- ▶ This is based on the assumption that processes tend to use only a small part of their address spaces while executing.
- ▶ The process 's address space is left behind on its source node, and a **page of the migrant process's address space** is transferred from its source node to its destination node only when referenced.



3. Message - Forwarding mechanisms

- ▶ In moving a process, it must be ensured that all **pending, en-route, and future messages** arrive at the process's new location.
- ▶ The messages to be forwarded to the migrant process's new location can be classified into the following:

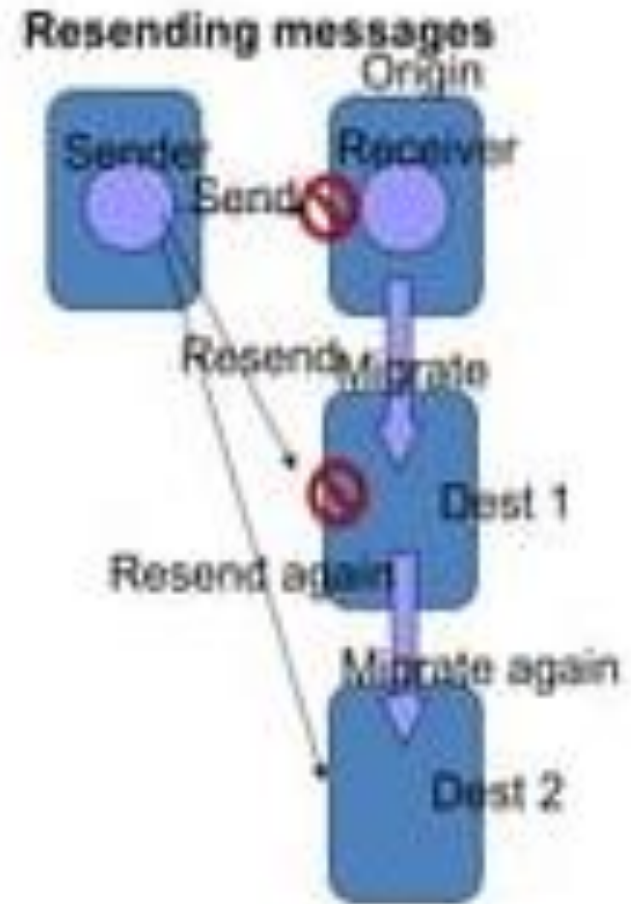
Type 1: Messages received at the source node after the process's execution has been stopped on its source node and the process's execution has not yet been started on its destination node

Type 2: Messages received at the source node after the process's execution has started on its destination node

Type 3: Messages that are to be sent to the migrant process from any other node after it has started executing on the destination node

3. Message - Forwarding mechanisms

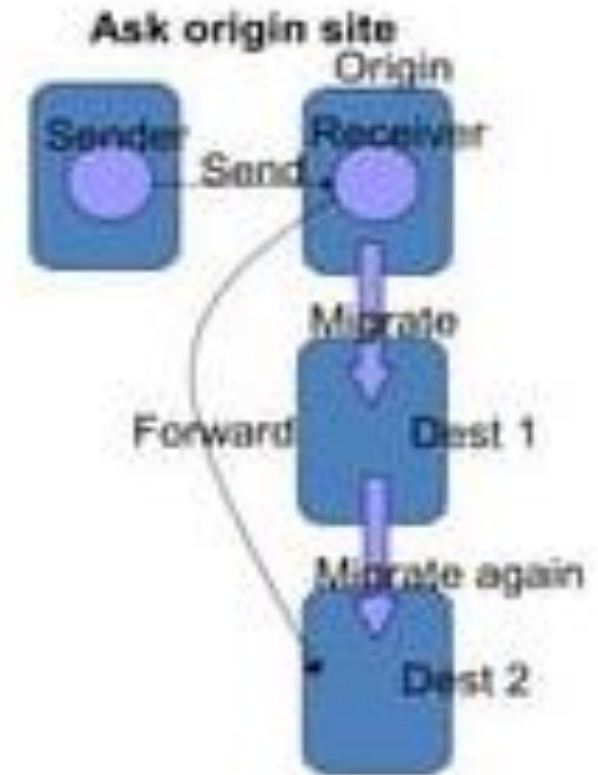
- ▶ Mechanisms used for message forwarding in existing DS to handle messages of all three types are:
- ▶ ***Mechanism of Resending the Message:***
 - Messages of types 1 and 2 are returned to the sender as not deliverable or are simply dropped, with the assurance that the sender of the message is storing a copy of the data and is prepared to retransmit it.
 - Messages of type 3 are sent directly to the process's destination node



3. Message - Forwarding mechanisms

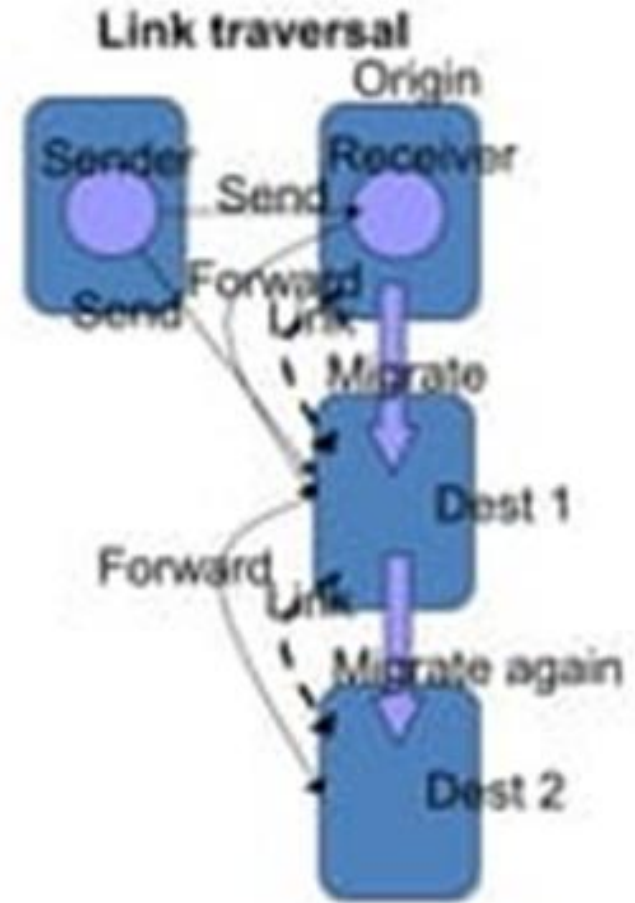
► *Origin Site Mechanism:*

- Messages for a particular process are always first sent to its origin site by using process identifier information.
- The origin site then forwards the message to the process's current location.
- Disadvantage: failure of the origin site disrupt the message forwarding mechanism.



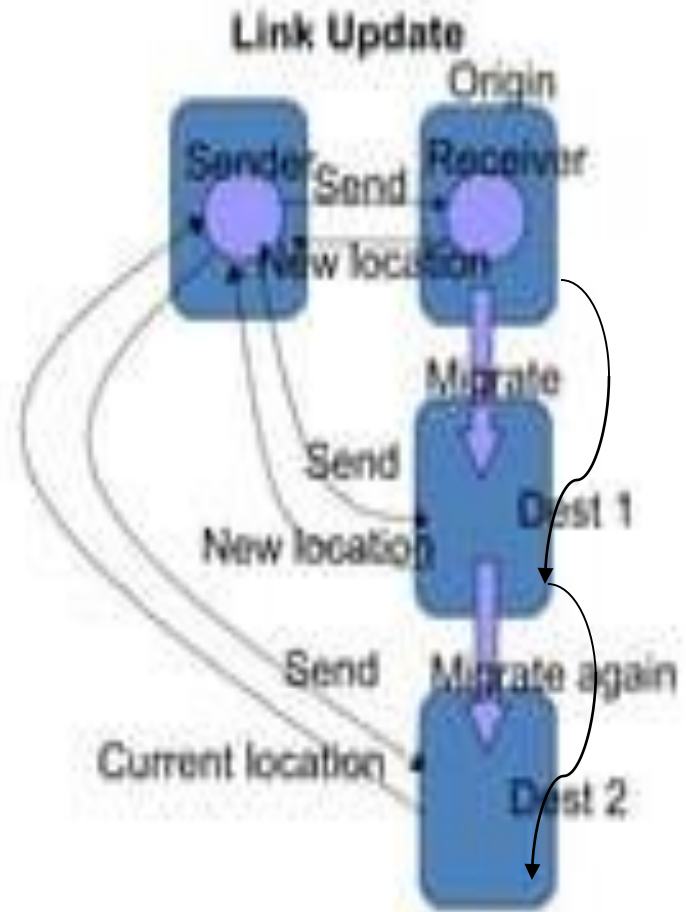
▶ **Link Traversal Mechanism:**

- To redirect the messages of type 1, a message queue for the migrant process is created on its source node and all the messages of this type are placed in this message queue.
- After the process is established on the destination node, all messages in the queue are sent to the destination node.
- To redirect the messages of types 2 and 3, a forwarding address known as *link* is left at the source node pointing to the destination node of the migrant process.
- Disadvantage: Several link may have to be traversed to locate a node and if any node in chain of link fails, the process cannot be located.



▶ **Link Update Mechanism:**

- During the transfer phase of the migrant process, the source node sends link-update messages to the kernels controlling all of the migrant process's communication partners.
- These link update messages tell the new address of each link held by the migrant process.
- Messages of types 1 and 2 are forwarded to the destination node by the source node and messages of type 3 are sent directly to the process's destination node



4. Mechanisms for Handling coprocesses

- ▶ Different mechanisms used for handling co-processes:

- ▶ **Disallowing separation of Coprocesses**

1. By disallowing the migration of processes that wait for one or more of their children to complete.
2. By ensuring that when a parent process migrates, its children processes will be migrated along with it.

- ▶ **Home Node or Origin Site concept**

1. All communications between a parent process and its children processes take place via the home node.
2. The message traffic & the communication cost increase considerably

Advantages of Process Migration

- ▶ Reducing average response time of processes
- ▶ Speeding up individual jobs
- ▶ Gaining higher throughput
- ▶ Utilizing resources effectively
- ▶ Reducing network traffic
- ▶ Improving system reliability
- ▶ Improving system security

CODE Migration

- ▶ Key reasons: performance and flexibility
 - ▶ Process migration (aka strong mobility)
 - Improved system-wide performance – better utilization of system-wide resources
 - Examples: Condor, Data Quality Services (DQS)
 - ▶ Code migration (aka weak mobility)
 - Shipment of server code to client – filling forms (reduce communication, no need to pre-link stubs with client)
 - Ship parts of client application to server instead of data from server to client (e.g., databases)
 - Improve parallelism – agent-based web searches
- 

CODE MIGRATION

- ▶ Passing programs, even while they are being executed, **simplifies the design** of a distributed system.
- ▶ Code migration in distributed systems took place in the form of **process migration** in which an entire process was moved from one machine to another.
- ▶ Moving a running process to a different machine is a costly task. But it gives better **performance**.
- ▶ Performance can be improved when processes are moved from heavily-loaded to lightly-loaded nodes.

Approaches to Code Migration

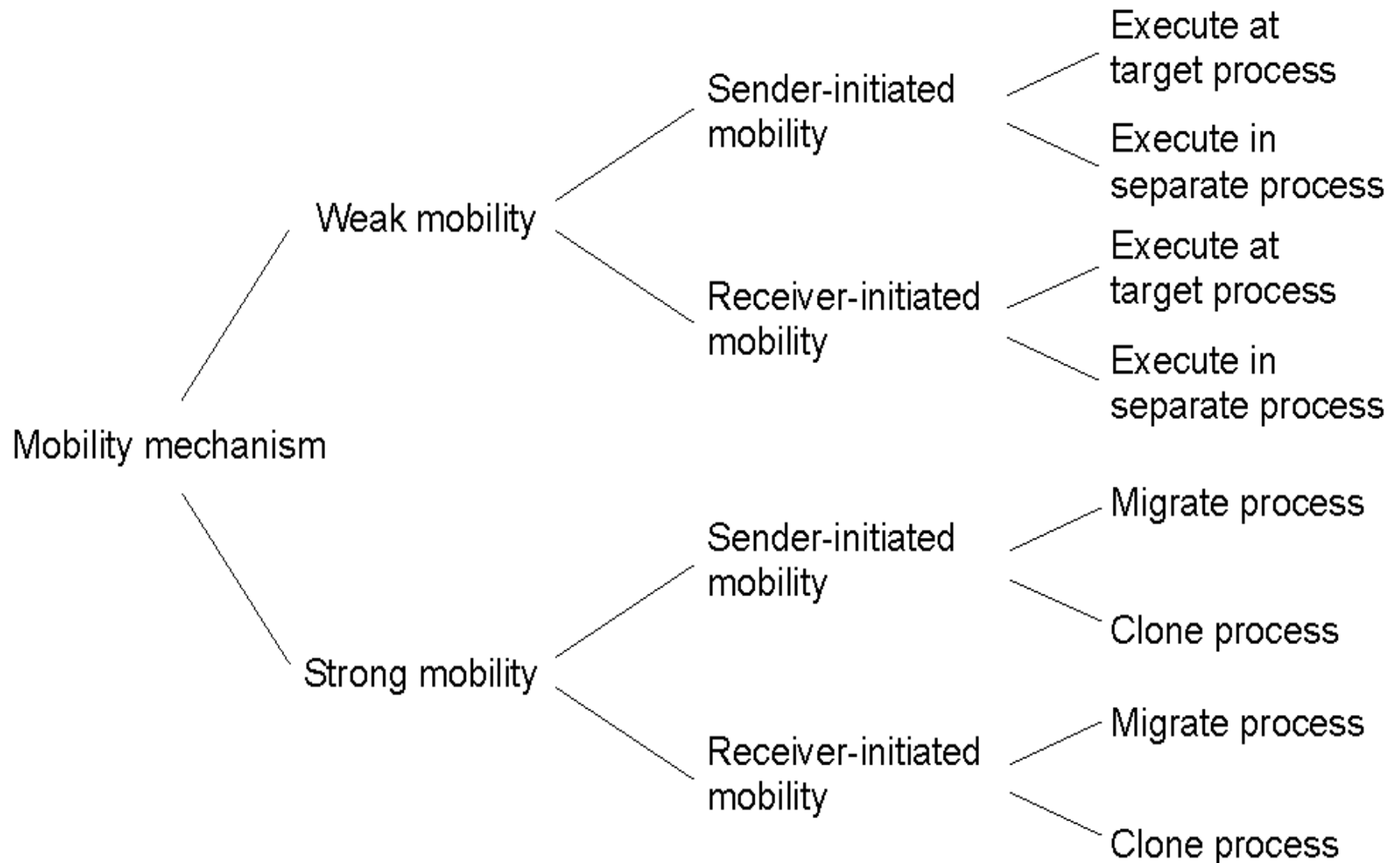
- ▶ Eg., A **client-server** system in which the server manages a huge database.
 - ▶ If a client application needs to perform many database operations involving large quantities of data, it may be better to ship part of the client application to the server and send only the results across the network.
 - ▶ Otherwise, the network maybe swamped with the transfer of data from the server to the client.
- ▶ In this case, code migration is based on the assumption that it generally makes sense to process data close to where those data reside.
- ▶ This same can be used for migrating parts of the **server to the client**.
 - ▶ Eg. In many interactive database applications, clients need to fill in forms that are subsequently translated into a series of database operations.
 - ▶ Processing the form at the client side, and sending only the completed form to the server, can avoid that a relatively large number of small messages need to cross the network.

CODE MIGRATION

A process consists of three segments to migrate.

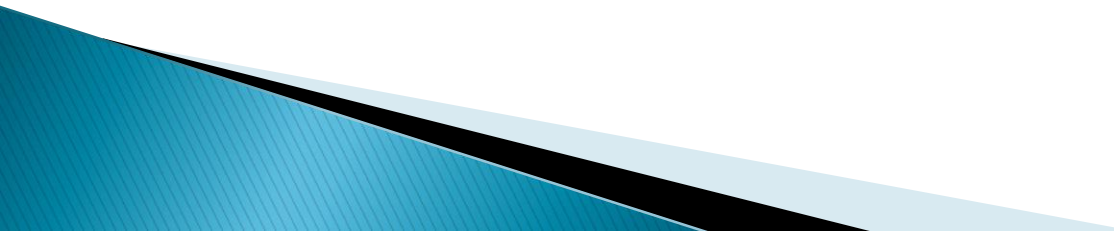
- ▶ The **code segment** is the part that contains the set of instructions that make up the program that is being executed.
- ▶ The **resource segment** contains references to external resources needed by the process, such as files, printers, devices, other processes, and so on.
- ▶ An **execution segment** is used to store the current execution state of a process, consisting of private data, the stack and the program counter.

Models for Code Migration



Mobility Mechanisms

1) Weak mobility:

- A characteristics feature of weak mobility is that a transferred program along with some initialization data and is always started from its initial state.
 - Weak mobility, also makes a difference if the migrated code is executed by the target process, or whether a separate process is started.
 - Eg., Java applets are simply downloaded by a Web browser and are executed in the browser's address space.
 - The benefit of this approach is that there is no need to start a separate process, thereby avoiding communication at the target machine.
- 

Mobility Mechanisms

2) Strong mobility

- A characteristics feature of strong mobility is that a running process can be stopped, subsequently moved to another m/c and then resume execution where it left off.
- Eg. D' Agents. (It is a mobile-agent system that is used primarily for distributed information-retrieval)
- Strong mobility can be supported by remote cloning. Cloning yields an exact copy of the original process, but now running on a different machine.
- The cloned process is executed in parallel to the original process.
- The benefit of cloning is that the model closely reassembles the one that is already used in many applications.

3) Sender initiated migration

- ▶ Migration is initiated at the m/c where code currently resides or is being executed.
- ▶ It is done when uploading programs to a compute server.
- ▶ Eg. Sending a search program access the internet to a web database server to perform the queries at that server.

4) Receiver initiated migration

- ▶ Where the initiative for code migration is taken by the target machine.
- ▶ Code migration occurs between a client and a server, where the client takes the initiative for migration.
- ▶ Securely uploading code to a server, as is done in **sender-initiated migration**, often requires that the client has previously been registered and authenticated at that server.
- ▶ Downloading code as in the **receiver-initiated case**, to the client is done only for improving client-side performance.